



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Motor de Integridad Multibase NoSQL

Curso: Base de Datos II

Docente: Ing. Patrick José Cuadros Quiroga

Integrantes:

Salas Acuña, Adrian Fabricio ***(2021069829)***
Lopez Ojeda, Keyla Noelia ***(2023078690)***

Tacna – Perú
2026



Datastack



CloudNexus

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	KLO / ASA	ASA	KLO	01/10/2026	Versión original



Datastack · CloudNexus S.A.C.

Sistema Motor de Integridad Multibase NoSQL Documento de Arquitectura de Software

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	KLO / ASA	ASA	KLO	01/10/2026	Versión original

ÍNDICE GENERAL

1. INTRODUCCIÓN	4
1.1. Propósito (Diagrama 4+1).....	4
1.2. Alcance.....	5
1.3. Definición, siglas y abreviaturas.....	5
1.4. Organización del documento	6
2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS.....	6
2.1. Priorización de requerimientos	6
2.1.1. Requerimientos Funcionales	6
2.1.2. Requerimientos No Funcionales – Atributos de Calidad	7
2.2. Restricciones	8
3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA.....	9
3.1. Vista de Caso de uso.....	9
3.1.1. Diagramas de Casos de uso	9
3.2. Vista Lógica	10
3.2.1. Diagrama de Subsistemas (paquetes).....	10
3.2.2. Diagrama de Secuencia (vista de diseño)	11
3.2.3. Diagrama de Colaboración (vista de diseño)	13
3.2.4. Diagrama de Objetos	14
3.2.5. Diagrama de Clases.....	15
3.2.6. Diagrama de Base de datos (relacional o no relacional).....	17
3.3. Vista de Implementación (vista de desarrollo).....	19
3.3.1. Diagrama de arquitectura software (paquetes).....	20
3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)	22
3.4. Vista de procesos	23
3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)	23
3.5. Vista de Despliegue (vista física).....	24
3.5.1. Diagrama de despliegue.....	25
4. ATRIBUTOS DE CALIDAD DEL SOFTWARE	26
Escenario de Funcionalidad	26
Escenario de Usabilidad	26
Escenario de confiabilidad.....	27
Escenario de rendimiento	27
Escenario de mantenibilidad.....	27
Otros Escenarios	28
REFERENCIAS	29

1. INTRODUCCIÓN

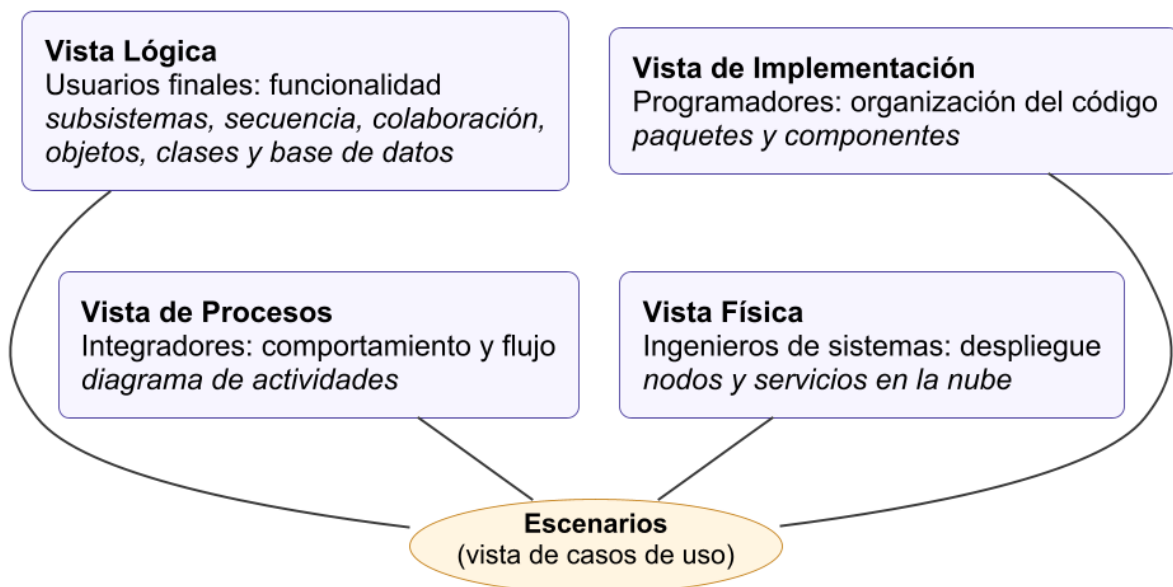
Este documento describe la arquitectura de software del *Motor de Integridad Multibase NoSQL*, desarrollado por Datastack para CloudNexus S.A.C. Traduce los requerimientos especificados en el SRS (FD03) en decisiones de diseño, componentes, relaciones y escenarios de despliegue, y sirve de guía para la construcción, las pruebas y el mantenimiento del sistema.

1.1. Propósito (Diagrama 4+1)

El propósito del documento es presentar una visión global de la arquitectura y de los objetivos de su diseño. Para ello se utiliza el modelo de vistas 4+1 propuesto por Kruchten (1995), que describe el sistema desde cuatro vistas complementarias, la lógica, la de implementación, la de procesos y la física, integradas por una quinta vista de escenarios o casos de uso que las valida (Figura 1).

Figura 1

Modelo de Vistas 4+1 Aplicado al Motor de Integridad



Nota. Adaptado de Kruchten (1995). Elaboración propia con PlantUML (The PlantUML Team, s.f.).

La arquitectura combina tres estilos: el patrón Modelo-Vista-Controlador para separar la interfaz, el control y la lógica de negocio; una arquitectura en capas con una capa

de integración basada en el patrón Adaptador, que permite tratar a MongoDB, Redis y Cassandra con una misma interfaz (Gamma et al., 1994); y un plano de control centralizado en Supabase que almacena usuarios, reglas, conexiones cifradas y auditoría.

1.2. Alcance

El documento abarca la versión 1.0 del sistema y describe las cinco vistas del modelo 4+1: la vista de casos de uso, con los casos arquitectónicamente significativos; la vista lógica, con los subsistemas, la interacción entre objetos, las clases y el modelo de datos relacional y NoSQL; la vista de implementación, con la organización del código y los componentes; la vista de procesos, con el flujo de actividades del sistema; y la vista física, con el despliegue en Streamlit Community Cloud y los servicios gestionados. Finalmente, se definen los escenarios de los atributos de calidad.

1.3. Definición, siglas y abreviaturas

La Tabla 1 reúne los términos propios de la arquitectura. Las definiciones funcionales se encuentran en el documento de visión (FD02).

Tabla 1

Definiciones, Siglas y Abreviaturas

Término	Definición
4+1	Modelo de vistas arquitectónicas de Kruchten (1995).
Adaptador	Patrón de diseño que traduce una interfaz común a la API de cada motor NoSQL.
BaaS / DBaaS	Backend o base de datos ofrecida como servicio gestionado en la nube.
Fernet	Cifrado simétrico autenticado (AES-128 en modo CBC con HMAC-SHA256) de la biblioteca cryptography.
MVC	Patrón Modelo-Vista-Controlador.
PaaS	Plataforma como servicio; en este proyecto, Streamlit Community Cloud.
Plano de control	Base de datos de configuración del sistema (Supabase).
QA	<i>Quality attribute</i> : atributo de calidad medible del sistema.
RLS	<i>Row Level Security</i> : seguridad a nivel de fila de PostgreSQL.
Secure Connect Bundle	Archivo de DataStax Astra con certificados para conectarse por TLS.
Singleton	Patrón que garantiza una única instancia (cliente de Supabase).
TLS / HTTPS	Protocolos de comunicación cifrada.

Término	Definición
UML	Lenguaje Unificado de Modelado (Object Management Group [OMG], 2017).

Nota. Elaboración propia.

1.4. Organización del documento

El capítulo 2 establece los objetivos y restricciones arquitectónicas mediante la priorización de requerimientos. El capítulo 3 representa la arquitectura en las vistas de casos de uso, lógica, de implementación, de procesos y de despliegue. El capítulo 4 define los escenarios de los atributos de calidad que la arquitectura debe satisfacer. Al final se listan las referencias.

2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS

Los objetivos arquitectónicos se derivan de los requerimientos del SRS (FD03). Se priorizan para definir el orden de implementación y para identificar los requerimientos que más condicionan el diseño.

2.1. Priorización de requerimientos

Los requerimientos se clasifican en tres niveles de prioridad: alta, para los indispensables del producto mínimo; media, para los que completan la experiencia del usuario; y baja, para los complementarios. Con esta prioridad se definió el orden de implementación en tres iteraciones.

2.1.1. Requerimientos Funcionales

Tabla 2

Priorización de Requerimientos Funcionales

ID	Descripción	Prioridad
RF01	Registrar cuenta. Crear una cuenta con usuario único (3 a 30 caracteres, sin distinguir mayúsculas) y contraseña de al menos 6 caracteres protegida con bcrypt.	Alta
RF02	Iniciar sesión. Autenticar al usuario con su usuario y contraseña y abrir su espacio de trabajo.	Alta
RF03	Cerrar sesión. Finalizar la sesión y volver a la pantalla de acceso.	Media
RF04	Registrar conexión. Registrar o actualizar la conexión a MongoDB, Redis o Cassandra, probarla y guardarla cifrada con Fernet.	Alta

ID	Descripción	Prioridad
RF05	Eliminar conexión. Eliminar la conexión guardada de un motor.	Media
RF06	Crear regla. Crear una regla de esquema, unicidad, integridad referencial o consistencia, con motor, colección y campo de origen y, si corresponde, de destino o JSON Schema.	Alta
RF07	Listar reglas. Mostrar las reglas activas del usuario.	Alta
RF08	Eliminar regla. Eliminar una regla junto con sus violaciones registradas.	Media
RF09	Validar registro. Validar un registro JSON contra las reglas de esquema y unicidad de su colección antes de escribirlo e informar el resultado.	Alta
RF10	Ejecutar escaneo por lotes. Recorrer los datos de los motores y evaluar todas las reglas activas del usuario.	Alta
RF11	Registrar violaciones. Guardar cada violación con su regla, tipo, motores involucrados, dato afectado y fecha, reemplazando las de la ejecución anterior de la misma regla.	Alta
RF12	Mostrar dashboard. Mostrar los motores utilizados, la cantidad de reglas y de anomalías y su distribución.	Media
RF13	Diagnosticar motor. Mostrar el estado de conexión, la latencia, la versión y los recursos de un motor.	Media
RF14	Explorar datos del motor. Mostrar esquemas, una muestra de registros y las estadísticas del servidor.	Baja
RF15	Generar reporte de anomalías. Listar las violaciones con su explicación y una sugerencia de corrección.	Media
RF16	Exportar reporte PDF. Descargar el reporte de anomalías en formato PDF.	Media
RF17	Mostrar usuarios en línea. Mostrar los usuarios con actividad en los últimos cinco minutos.	Baja

Nota. Elaboración propia.

2.1.2. Requerimientos No Funcionales – Atributos de Calidad

Tabla 3

Priorización de Requerimientos No Funcionales

ID	Descripción	Prioridad
RNF01	Seguridad. Las contraseñas se almacenan con hash bcrypt y nunca en texto plano.	Alta
RNF02	Seguridad. Las credenciales de los motores se almacenan cifradas con Fernet y las tablas de Supabase tienen seguridad a nivel de fila (RLS).	Alta
RNF03	Seguridad. Cada usuario accede únicamente a sus conexiones, reglas y violaciones.	Alta
RNF04	Comunicaciones. Toda comunicación usa HTTPS o TLS.	Alta

ID	Descripción	Prioridad
RNF05	Rendimiento. La validación de un registro responde en menos de 3 segundos.	Media
RNF06	Fiabilidad. Si una regla no puede evaluarse, el escaneo continúa con las demás y conserva sus violaciones anteriores.	Alta
RNF07	Usabilidad. La interfaz está en español y un usuario nuevo crea una regla y ejecuta un escaneo en menos de 10 minutos.	Media
RNF08	Compatibilidad. Funciona en las versiones actuales de Chrome, Edge y Firefox sin instalación.	Media
RNF09	Disponibilidad. La aplicación está disponible las 24 horas, sujeta a la disponibilidad de los proveedores en la nube.	Media
RNF10	Mantenibilidad. La arquitectura sigue el patrón MVC y el 100 % de las pruebas unitarias se aprueban antes de cada publicación.	Alta
RNF11	Extensibilidad. Se puede incorporar un nuevo motor implementando la interfaz Adaptador sin modificar el motor de reglas.	Media
RNF12	Portabilidad. La aplicación se despliega en Streamlit Community Cloud con Python 3.12.	Media

Nota. Elaboración propia.

Los atributos de calidad son propiedades medibles y evaluables del sistema que indican en qué grado satisface las necesidades de sus interesados (Bass et al., 2021). En este proyecto, la seguridad, la fiabilidad del escaneo y la extensibilidad hacia nuevos motores son los que más influyen en la arquitectura; por ello se reflejan en el cifrado de credenciales, en el manejo de errores por regla y en la capa de adaptadores.

2.2. Restricciones

- **Tecnológicas:** la aplicación se desarrolla en Python 3.12 con Streamlit y se despliega en Streamlit Community Cloud a partir del repositorio de GitHub.
- **De infraestructura:** se utilizan los planes gratuitos de Supabase, MongoDB Atlas, Upstash y DataStax Astra, con sus límites de almacenamiento y operaciones.
- **De seguridad:** las credenciales nunca se almacenan en texto plano ni en el repositorio; los secretos se configuran en la plataforma de despliegue.
- **De alcance:** la versión 1.0 soporta solo MongoDB, Redis y Apache Cassandra, y no corrige datos ni intercepta escrituras de terceros.

- **De tiempo y equipo:** dos desarrolladores durante 16 semanas, dentro del semestre académico 2026-II.

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

3.1. Vista de Caso de uso

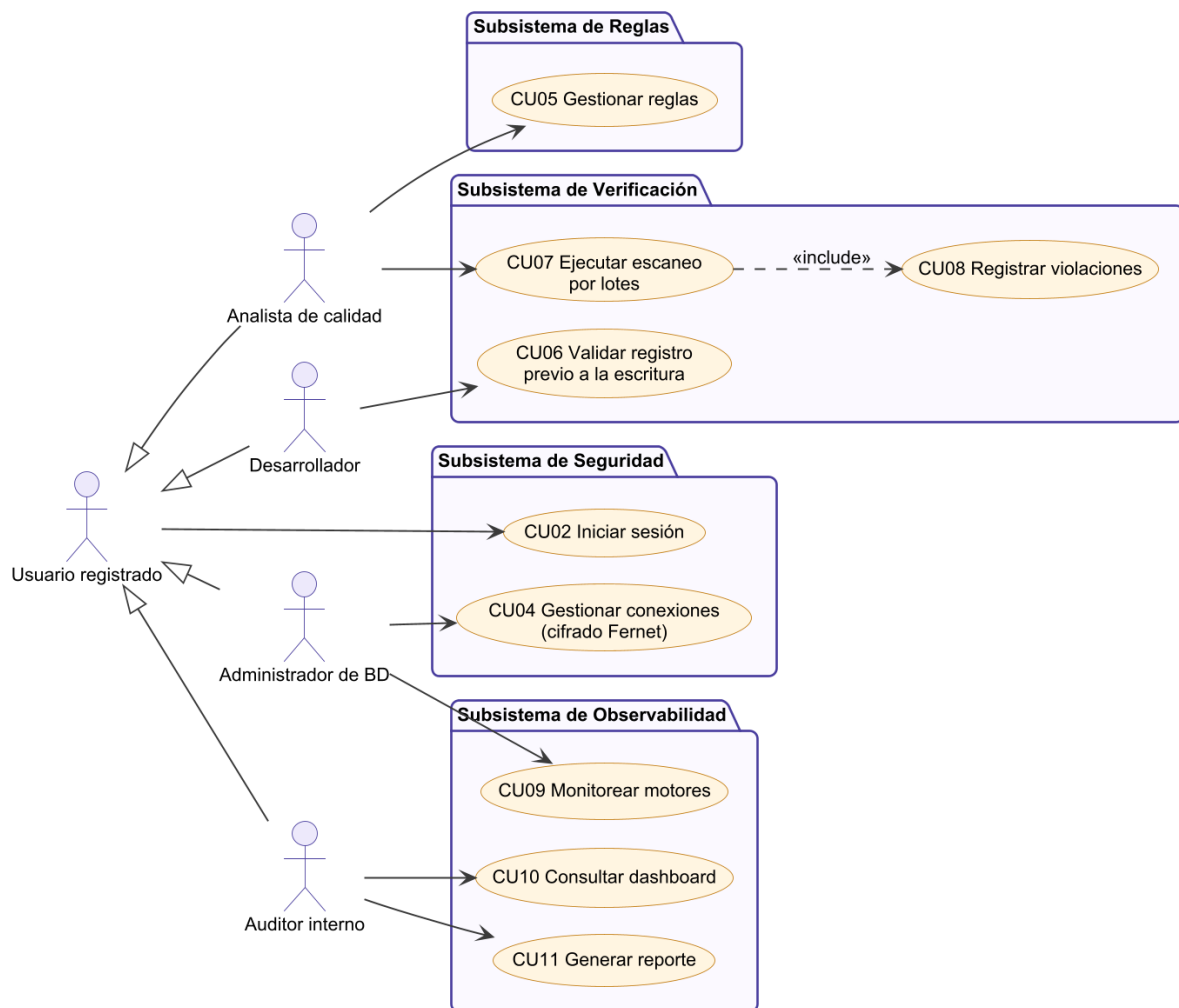
Esta vista describe los casos de uso que cubren las funcionalidades del sistema y valida las demás vistas. De los trece casos de uso del SRS, se consideran arquitectónicamente significativos los que atraviesan todas las capas o condicionan decisiones de diseño: la gestión de conexiones, que exige el cifrado y la fábrica de adaptadores; la validación previa y el escaneo por lotes, que ejercitan el motor de reglas sobre los tres motores; y el registro de violaciones, que define el modelo de auditoría.

3.1.1. Diagramas de Casos de uso

La Figura 2 agrupa los casos de uso significativos por subsistema. El diagrama completo de casos de uso y sus narrativas se encuentran en el SRS (FD03).

Figura 2

Casos de Uso Arquitectónicamente Significativos por Subsistema



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.2. Vista Lógica

La vista lógica representa los requerimientos funcionales mediante la descomposición del sistema en subsistemas, la interacción entre sus objetos, las clases de diseño y el modelo de datos.

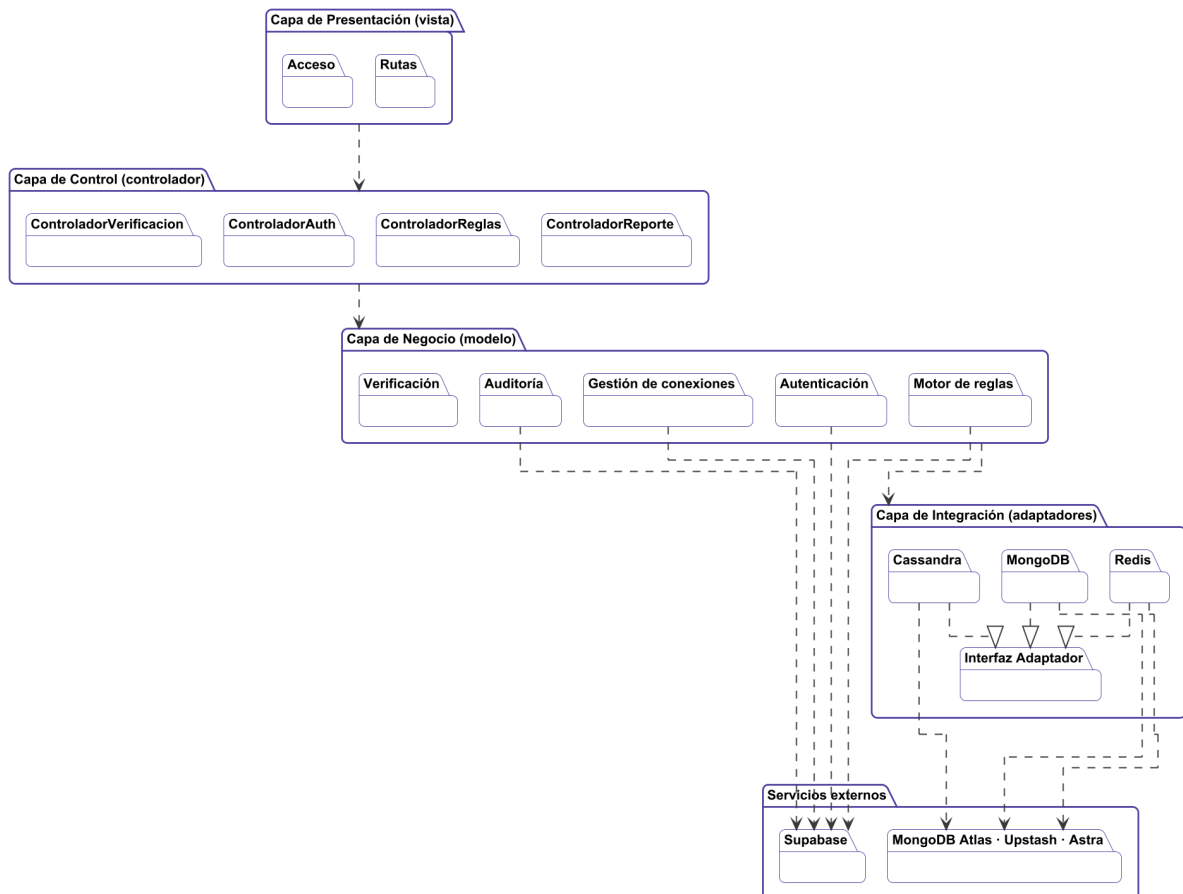
3.2.1. Diagrama de Subsistemas (paquetes)

El sistema se divide en cinco capas (Figura 3). Cada capa solo depende de la inmediatamente inferior: la vista invoca a los controladores, estos al modelo y el modelo a los adaptadores. Los tres adaptadores realizan la interfaz Adaptador, por lo que el modelo

no conoce los detalles de cada motor. Supabase y los motores NoSQL son servicios externos.

Figura 3

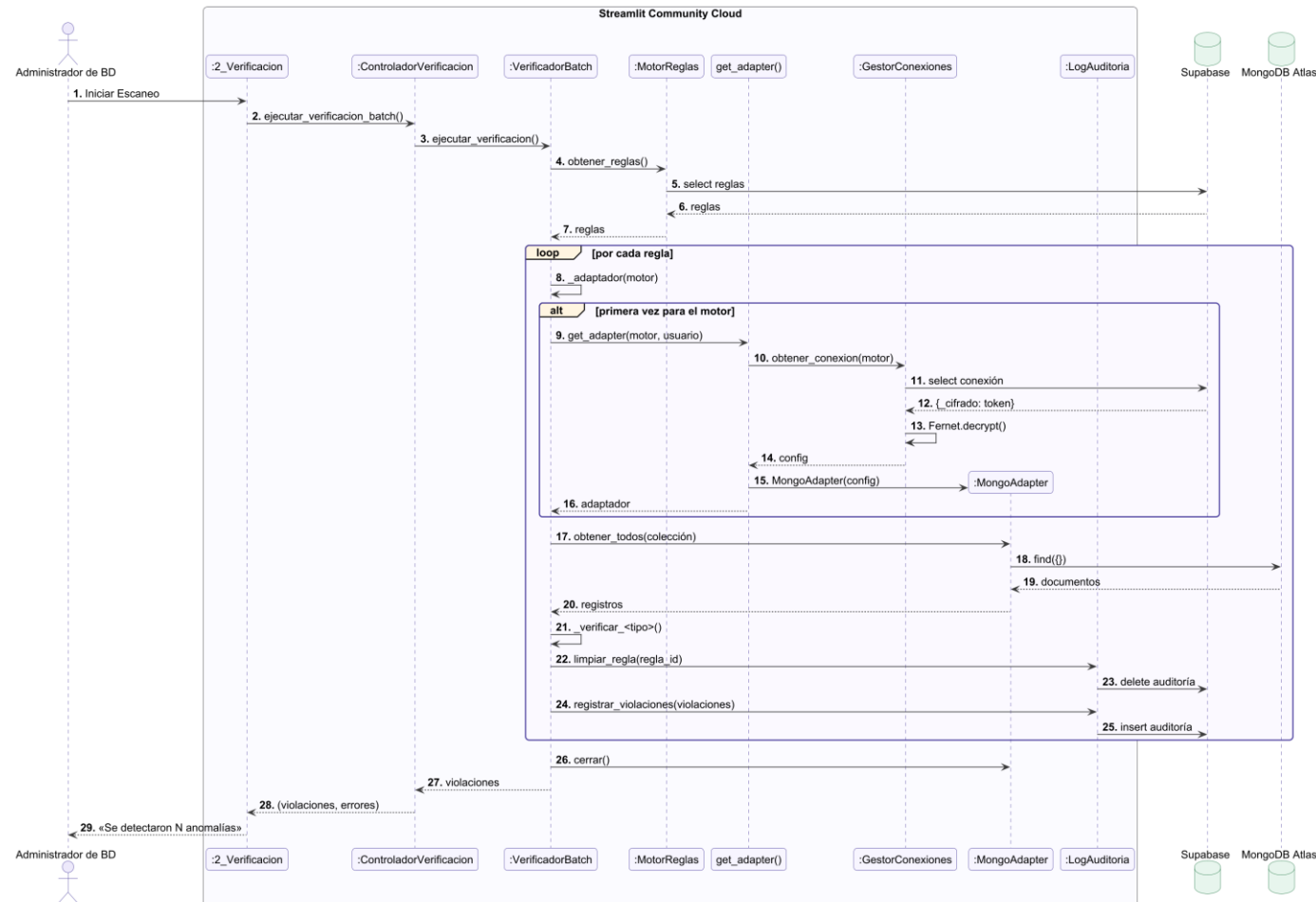
Diagrama de Subsistemas por Capas



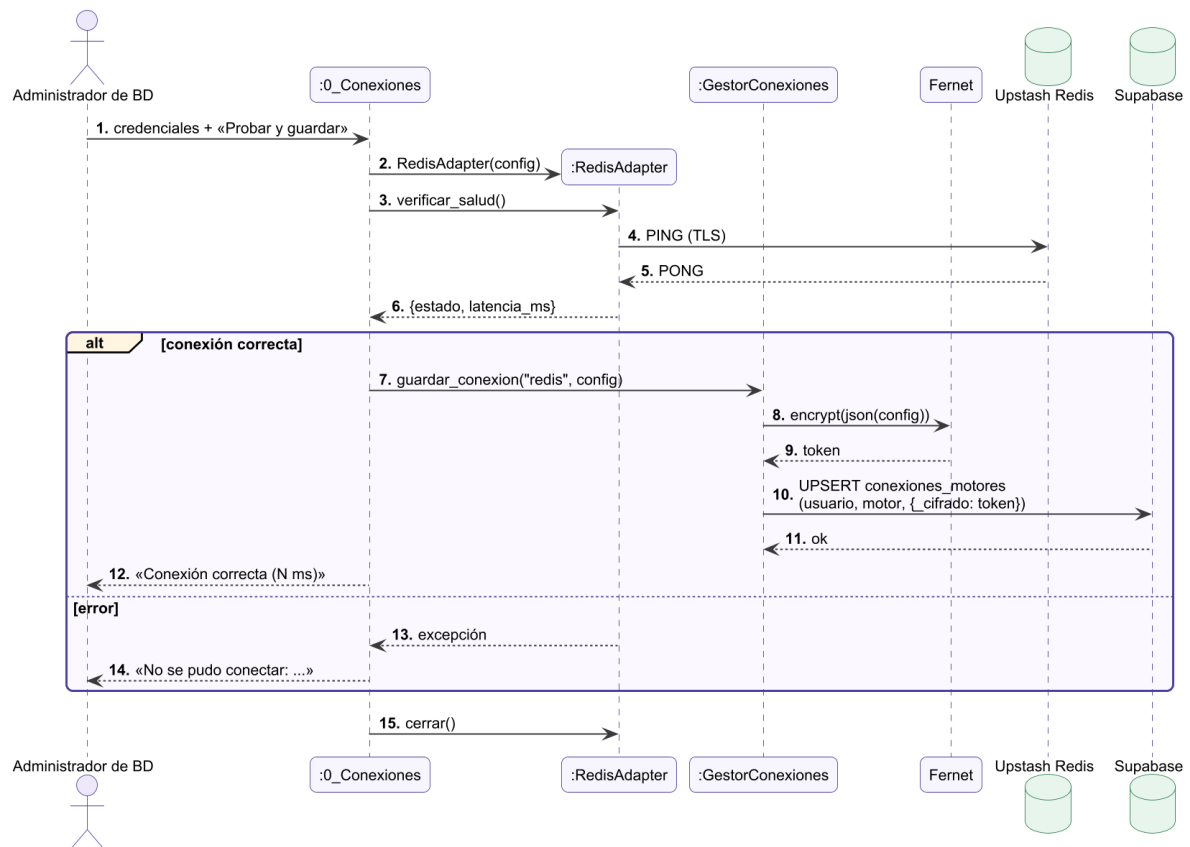
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.2.2. Diagrama de Secuencia (vista de diseño)

Las Figuras 4 y 5 muestran, con las clases reales del diseño, los dos escenarios más significativos. En el escaneo por lotes, el verificador reutiliza una conexión por motor, descifra las credenciales con Fernet, evalúa cada regla y reemplaza sus violaciones en la auditoría. En el registro de una conexión, la aplicación prueba la conexión antes de cifrarla y guardarla.

Figura 4
Diagrama de Secuencia de Diseño: Escaneo por Lotes


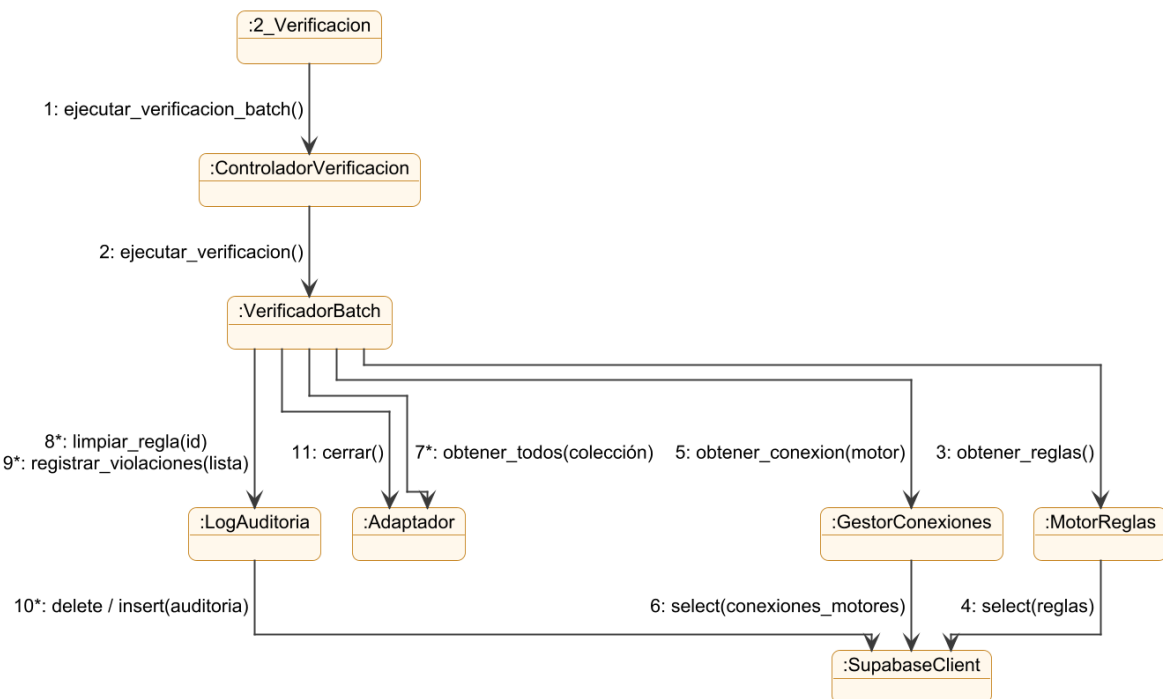
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

Figura 5
Diagrama de Secuencia de Diseño: Registrar una Conexión Cifrada


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.2.3. Diagrama de Colaboración (vista de diseño)

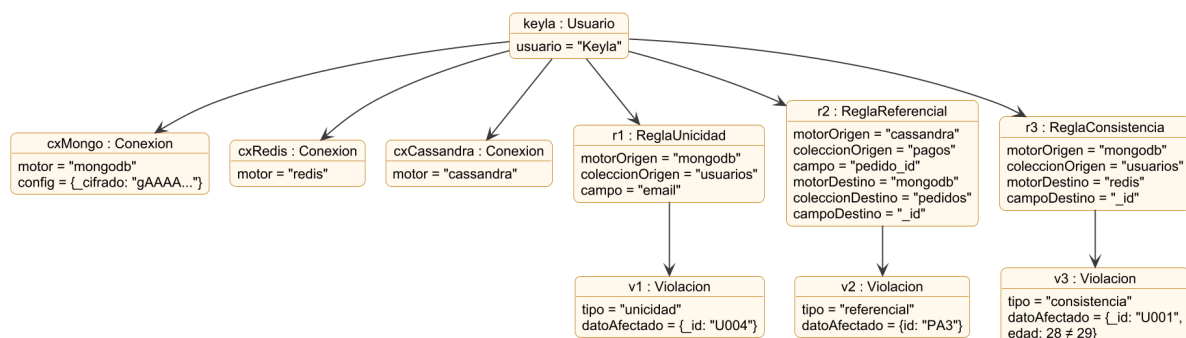
La Figura 6 presenta el mismo escenario de escaneo como diagrama de colaboración (comunicación), destacando los vínculos entre objetos y el orden de los mensajes. Los mensajes marcados con asterisco se repiten por cada regla.

Figura 6
Diagrama de Colaboración del Escaneo por Lotes


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.2.4. Diagrama de Objetos

La Figura 7 muestra una instantánea del sistema durante la verificación del conjunto de prueba: el usuario «Keyla» con sus tres conexiones, tres de sus reglas y las violaciones que estas generan, como el correo duplicado del usuario U004, el pago PA3 que referencia un pedido inexistente y la edad divergente del usuario U001 entre MongoDB y Redis.

Figura 7
Diagrama de Objetos con el Conjunto de Prueba


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.2.5. Diagrama de Clases

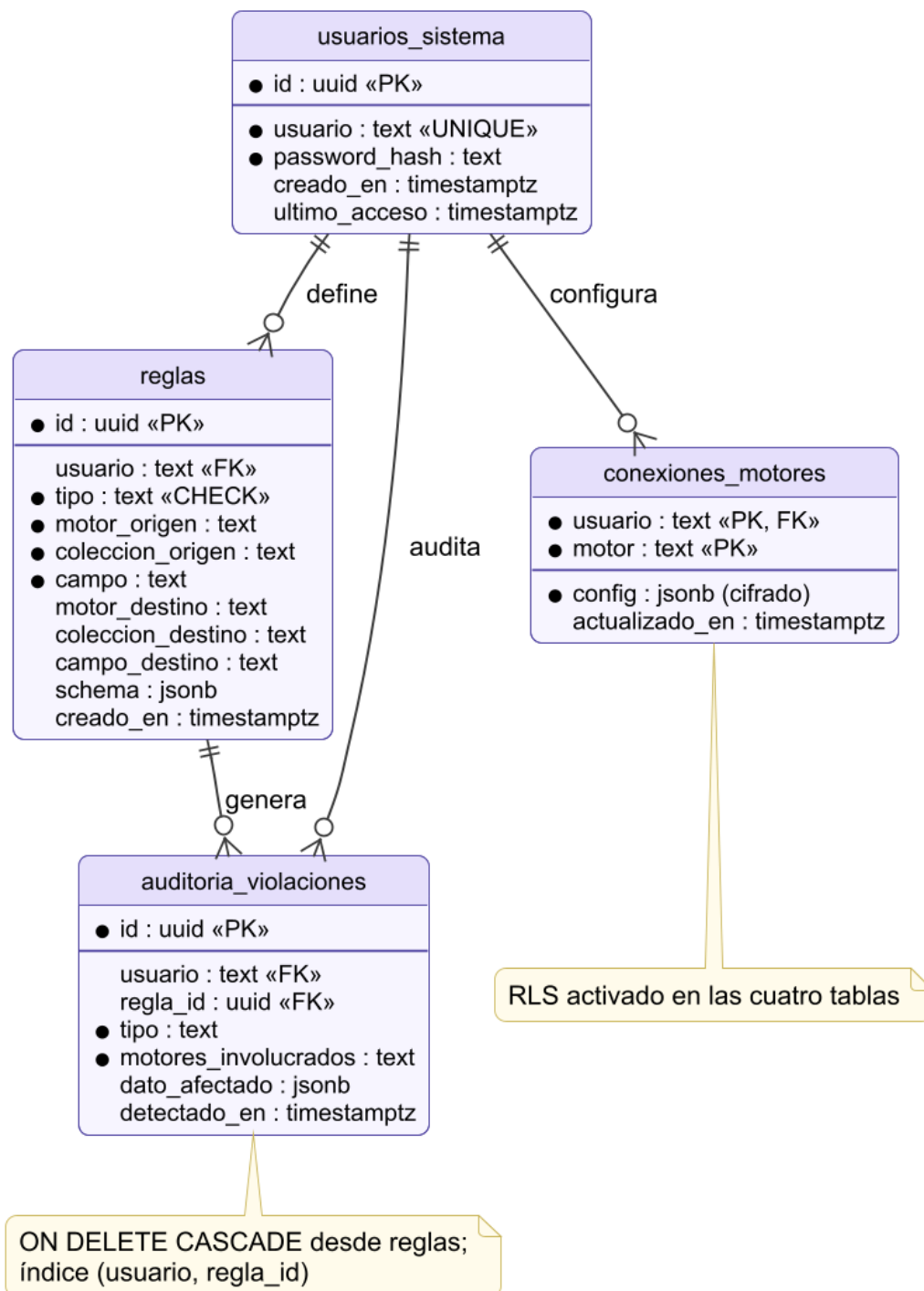
La Figura 8 presenta las clases de diseño organizadas por paquete. Destacan el cliente de Supabase como singleton, la clase abstracta Adaptador con sus tres implementaciones, la función fábrica `get_adapter`, que obtiene las credenciales descifradas y crea el adaptador correspondiente, y el `VerificadorBatch`, que implementa un método de verificación por cada tipo de regla.

Diagrama de Clases de Diseño



3.2.6. Diagrama de Base de datos (relacional o no relacional)

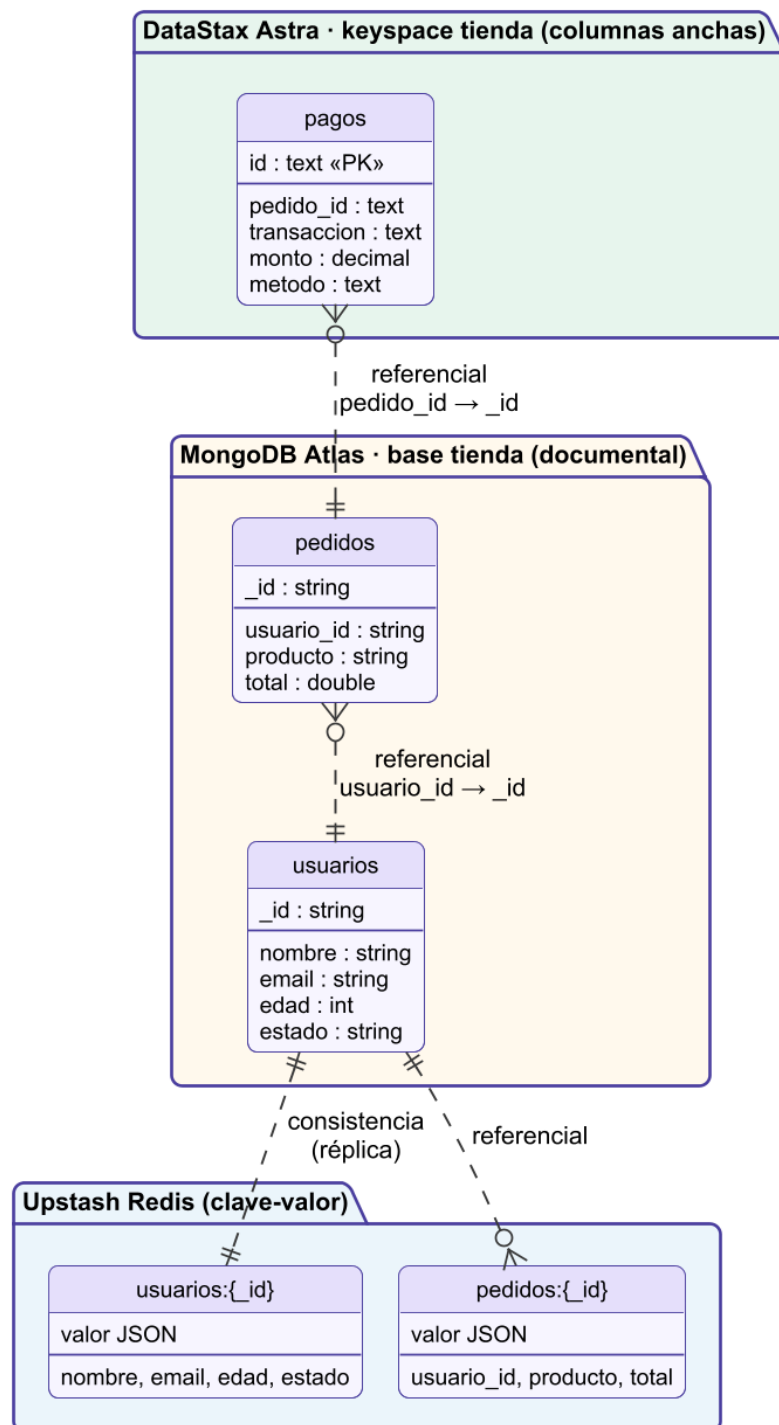
El sistema utiliza dos modelos de datos. El plano de control es relacional (PostgreSQL en Supabase) y consta de cuatro tablas con integridad referencial, eliminación en cascada y seguridad a nivel de fila (Figura 9). Los datos que se verifican son no relacionales y pertenecen al cliente; la Figura 10 muestra el modelo del conjunto de prueba y las reglas de integridad que los relacionan entre motores.

Figura 9
Modelo Relacional del Plano de Control en Supabase


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

Figura 10

Modelo de Datos NoSQL del Conjunto de Prueba y sus Reglas de Integridad



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.3. Vista de Implementación (vista de desarrollo)

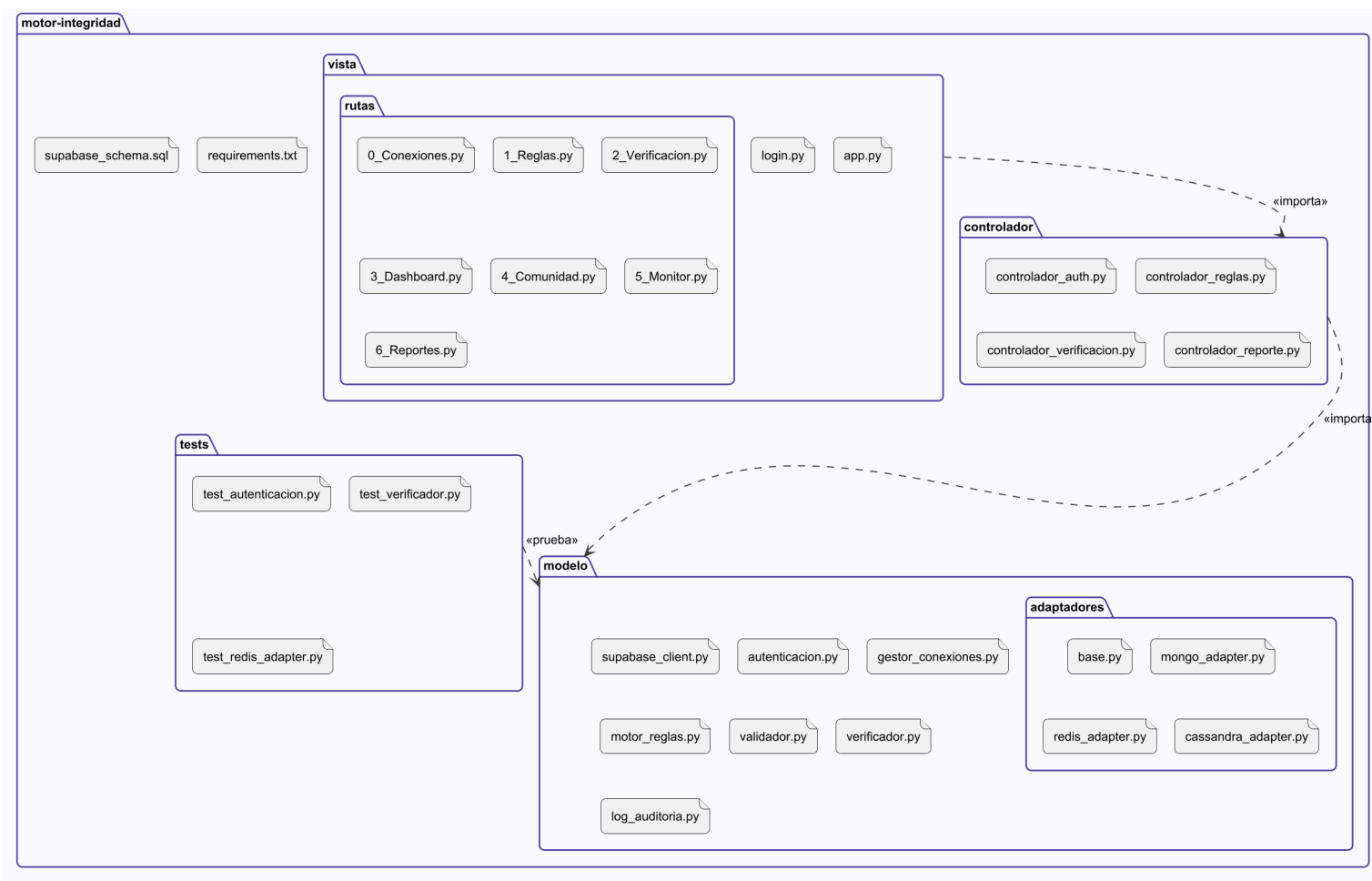
Esta vista describe la organización del código fuente en el repositorio y su mapeo con los subsistemas de la vista lógica.

3.3.1. Diagrama de arquitectura software (paquetes)

La Figura 11 muestra la estructura de paquetes y archivos del repositorio. La carpeta *vista* contiene la aplicación Streamlit y sus páginas; *controlador* y *modelo* implementan las capas de control y de negocio; *adaptadores* agrupa la integración con los motores; y *tests* contiene las pruebas unitarias con pytest.

Figura 11

Arquitectura de Software por Paquetes del Repositorio



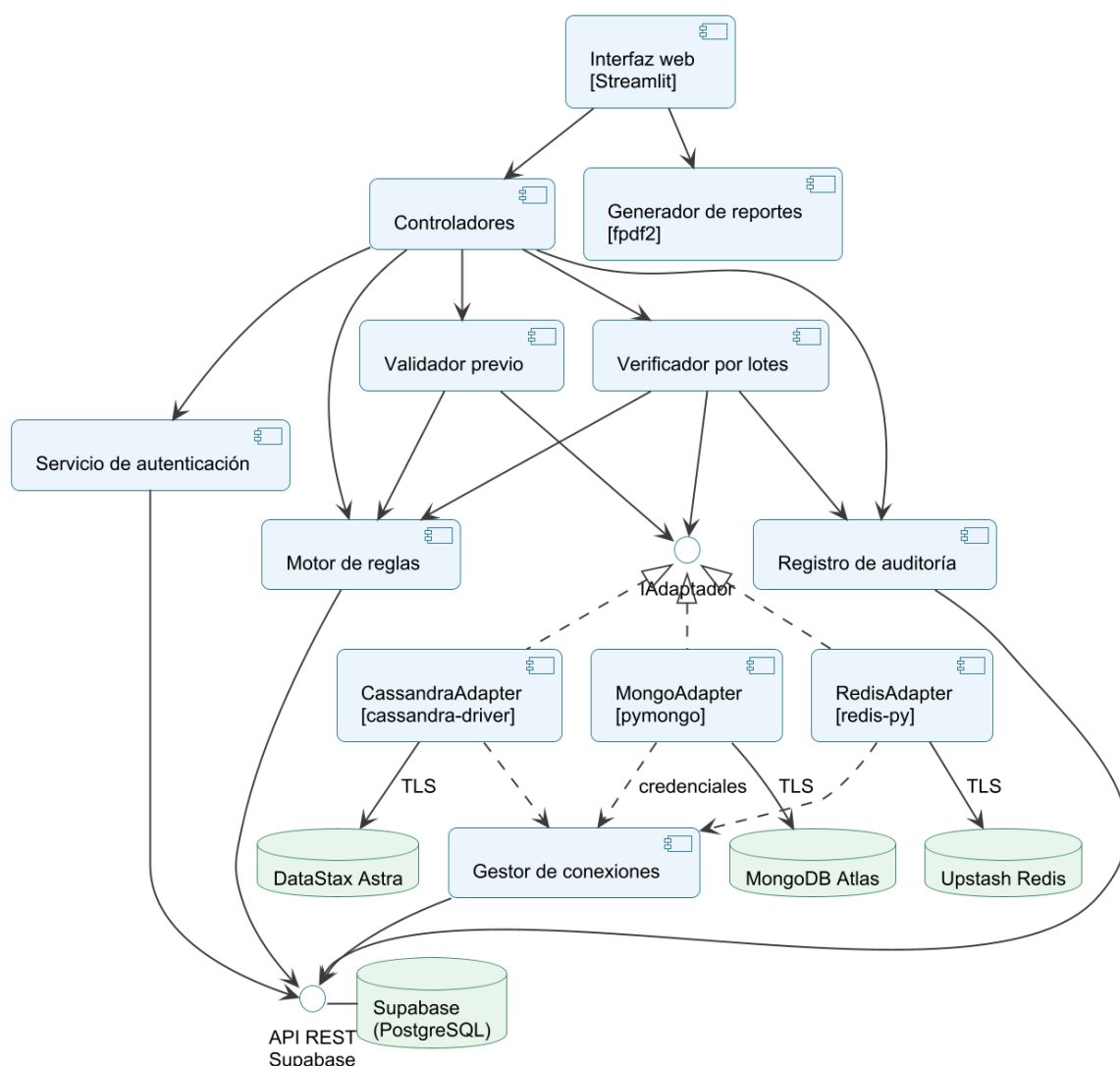
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)

La Figura 12 presenta los componentes del sistema y sus interfaces. Los componentes de validación y verificación dependen de la interfaz IAdaptador, realizada por los adaptadores de cada motor, que obtienen sus credenciales del gestor de conexiones; los servicios de autenticación, reglas, auditoría y conexiones usan la API REST de Supabase.

Figura 12

Diagrama de Componentes del Sistema



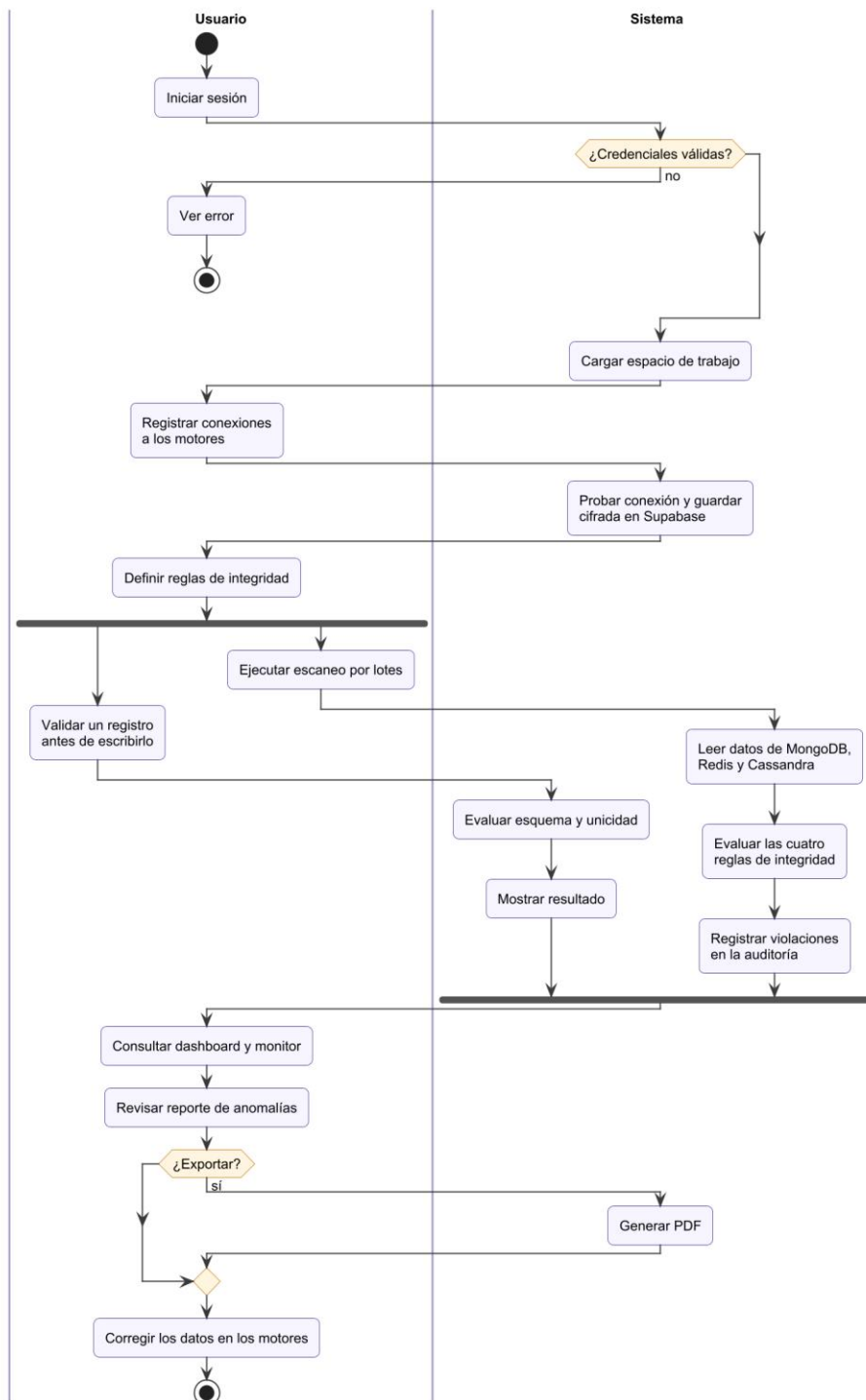
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.4. Vista de procesos

La aplicación se ejecuta como un proceso de Streamlit por sesión de usuario en el contenedor de Streamlit Community Cloud. Cada interacción vuelve a ejecutar la página correspondiente; las operaciones con los motores son síncronas y se agrupan por regla. El escaneo por lotes abre una sola conexión por motor durante toda la verificación y la cierra al final, mientras que el cliente de Supabase se comparte como singleton dentro del proceso.

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)

La Figura 13 muestra el proceso completo en el que interviene el sistema, desde el inicio de sesión hasta la consulta de reportes, incluidas las actividades paralelas de validación previa y escaneo por lotes.

Figura 13
Diagrama de Actividades del Proceso del Sistema


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

3.5. Vista de Despliegue (vista física)

El sistema se despliega íntegramente en la nube, sin servidores propios. El código se publica en GitHub y Streamlit Community Cloud lo redespiega automáticamente con

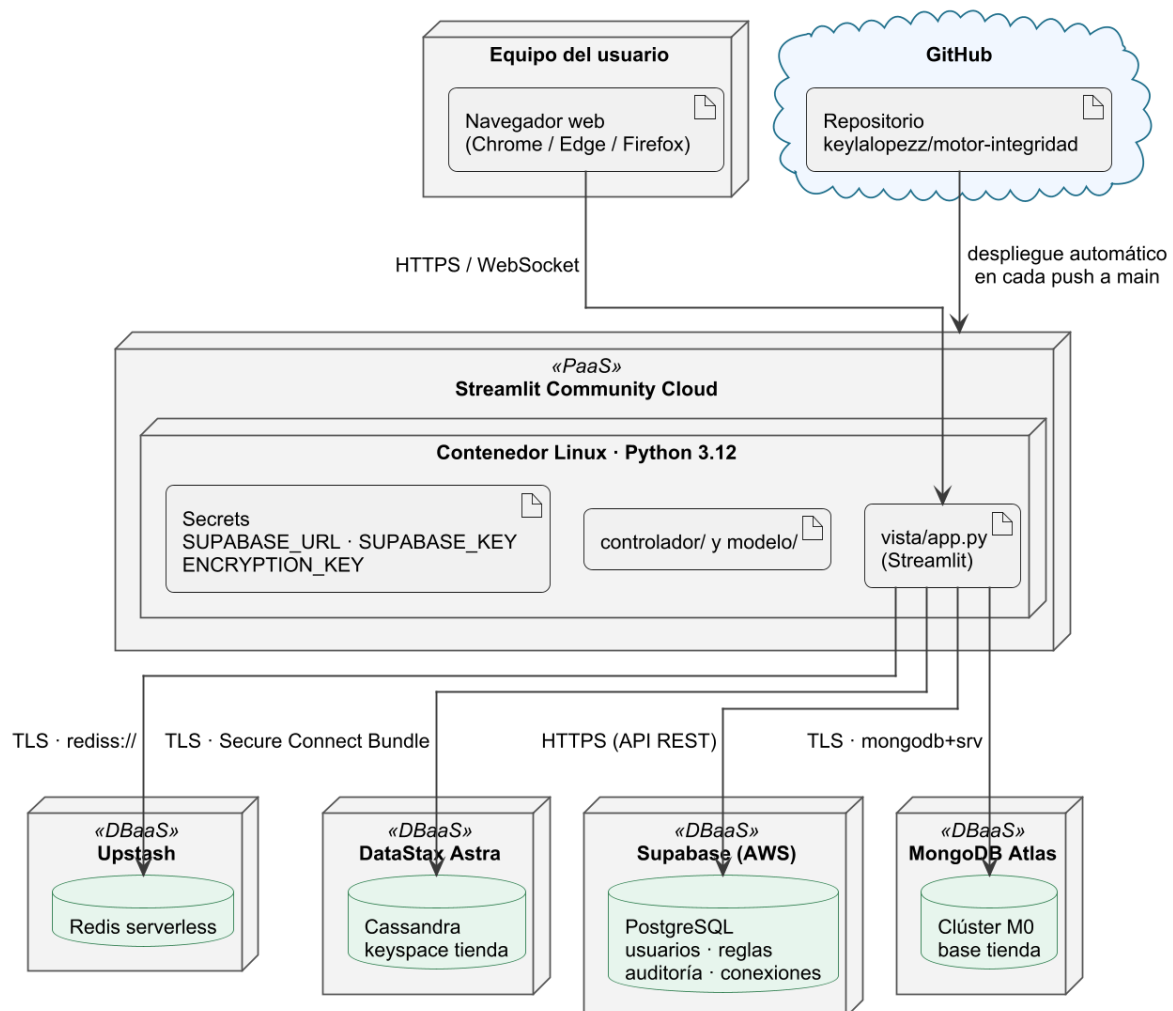
cada cambio en la rama principal. Los secretos (URL y clave de Supabase y clave de cifrado) se configuran en la plataforma y nunca en el repositorio.

3.5.1. Diagrama de despliegue

La Figura 14 muestra los nodos físicos y los protocolos de comunicación. El navegador se comunica con la aplicación por HTTPS; la aplicación accede a Supabase mediante su API REST y a los motores mediante conexiones TLS: *mongodb+srv* para MongoDB Atlas, *rediss* para Upstash y el Secure Connect Bundle para DataStax Astra.

Figura 14

Diagrama de Despliegue en la Nube



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

Los atributos de calidad se especifican mediante escenarios de seis partes: la fuente del estímulo, el estímulo, el artefacto afectado, el entorno, la respuesta y la medida de la respuesta (Bass et al., 2021). Cada escenario se vincula con los requerimientos no funcionales del SRS y con las características de calidad de la norma ISO/IEC 25010:2023 (International Organization for Standardization [ISO], 2023).

Escenario de Funcionalidad

Evalúa que el sistema detecte correctamente las violaciones de los cuatro tipos de regla (Tabla 4).

Tabla 4

Escenario de Funcionalidad

Fuente	Analista de calidad de datos.
Estímulo	Ejecuta el escaneo por lotes sobre el conjunto de prueba con siete violaciones inyectadas.
Artefacto	Verificador por lotes y adaptadores.
Entorno	Operación normal, con conexiones válidas a los tres motores.
Respuesta	El sistema detecta y registra las violaciones de esquema, unicidad, integridad referencial y consistencia.
Medida	100 % de las violaciones detectadas (7 de 7) y 0 % de falsos positivos.

Escenario de Usabilidad

Evalúa la facilidad con la que un usuario nuevo aprende a usar el sistema (Tabla 5).

Tabla 5

Escenario de Usabilidad

Fuente	Usuario nuevo de CloudNexus.
Estímulo	Desea crear una regla de unicidad y ejecutar un escaneo.
Artefacto	Interfaz web en Streamlit.
Entorno	Primer uso, después de una capacitación de cuatro horas.
Respuesta	Los módulos guían el flujo con formularios, mensajes en español y ayudas contextuales.
Medida	Completa la tarea en menos de 10 minutos sin asistencia (RNF07).

Escenario de confiabilidad

Evalúa el comportamiento ante fallas y la protección de la información (Tabla 6).

Tabla 6

Escenario de Confiabilidad

Fuente	Servicio externo (motor NoSQL).
Estímulo	Un motor deja de responder durante el escaneo por lotes.
Artefacto	Verificador por lotes.
Entorno	Escaneo en ejecución con reglas sobre varios motores.
Respuesta	Se registra el error de la regla afectada, se conservan sus violaciones anteriores y se continúa con las demás reglas; las conexiones se cierran al terminar.
Medida	0 escaneos interrumpidos por la falla de un motor y 0 pérdidas de violaciones previas (RNF06).

Escenario de rendimiento

Evalúa el tiempo de respuesta de las operaciones interactivas (Tabla 7).

Tabla 7

Escenario de Rendimiento

Fuente	Desarrollador.
Estímulo	Valida un registro en el simulador previo a la escritura.
Artefacto	Validador y adaptador del motor.
Entorno	Operación normal en la nube.
Respuesta	El sistema evalúa las reglas de esquema y unicidad y muestra el resultado.
Medida	Respuesta en menos de 3 segundos (RNF05); latencia de cada motor visible en el Monitor.

Escenario de mantenibilidad

Evalúa la facilidad para extender el sistema a nuevos motores (Tabla 8).

Tabla 8

Escenario de Mantenibilidad

Fuente	Desarrollador de Datastack.
Estímulo	Se solicita soportar un nuevo motor NoSQL.
Artefacto	Capa de adaptadores.
Entorno	Tiempo de desarrollo.

Respuesta	Se implementa una nueva clase que realiza la interfaz Adaptador y se registra en la fábrica, sin modificar el motor de reglas, el validador ni el verificador.
Medida	0 cambios en el modelo de negocio y 100 % de las pruebas unitarias aprobadas (RNF10, RNF11).

Otros Escenarios

Se definen, además, escenarios de seguridad y de portabilidad, por su impacto en la arquitectura (Tablas 9 y 10).

Tabla 9

Escenario de Seguridad

Fuente	Atacante externo con acceso a la clave pública (anon key) de Supabase.
Estímulo	Intenta leer las credenciales o los hashes de contraseñas.
Artefacto	Plano de control en Supabase.
Entorno	Sistema en producción.
Respuesta	RLS bloquea el acceso sin políticas públicas; aun así, las credenciales están cifradas con Fernet y las contraseñas, con bcrypt.
Medida	0 credenciales legibles y 0 secretos en el repositorio (RNF01 a RNF03).

Tabla 10

Escenario de Portabilidad

Fuente	Equipo de Datastack.
Estímulo	Publica una nueva versión en la rama principal del repositorio.
Artefacto	Aplicación completa.
Entorno	Streamlit Community Cloud.
Respuesta	La plataforma instala las dependencias de requirements.txt y redespiega la aplicación.
Medida	Nueva versión disponible en la URL pública en pocos minutos, sin pasos manuales (RNF12).

REFERENCIAS

Bass, L., Clements, P., y Kazman, R. (2021). *Software architecture in practice* (4.^a ed.).

Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC Standard No. 25010:2023).

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, 12(6), 42–50.

<https://doi.org/10.1109/52.469759>

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML)* (Versión 2.5.1). <https://www.omg.org/spec/UML/2.5.1>

The PlantUML Team. (s.f.). *PlantUML*. Recuperado el 3 de octubre de 2026, de

<https://plantuml.com/>